

# Informe de Refactorización: Sistema de Gestión de Transportes

## 1. Lista de Refactorizaciones Aplicadas

El proceso de mejora del código se ha llevado a cabo aplicando el catálogo estándar de refactorizaciones de Martin Fowler, en el siguiente orden lógico:

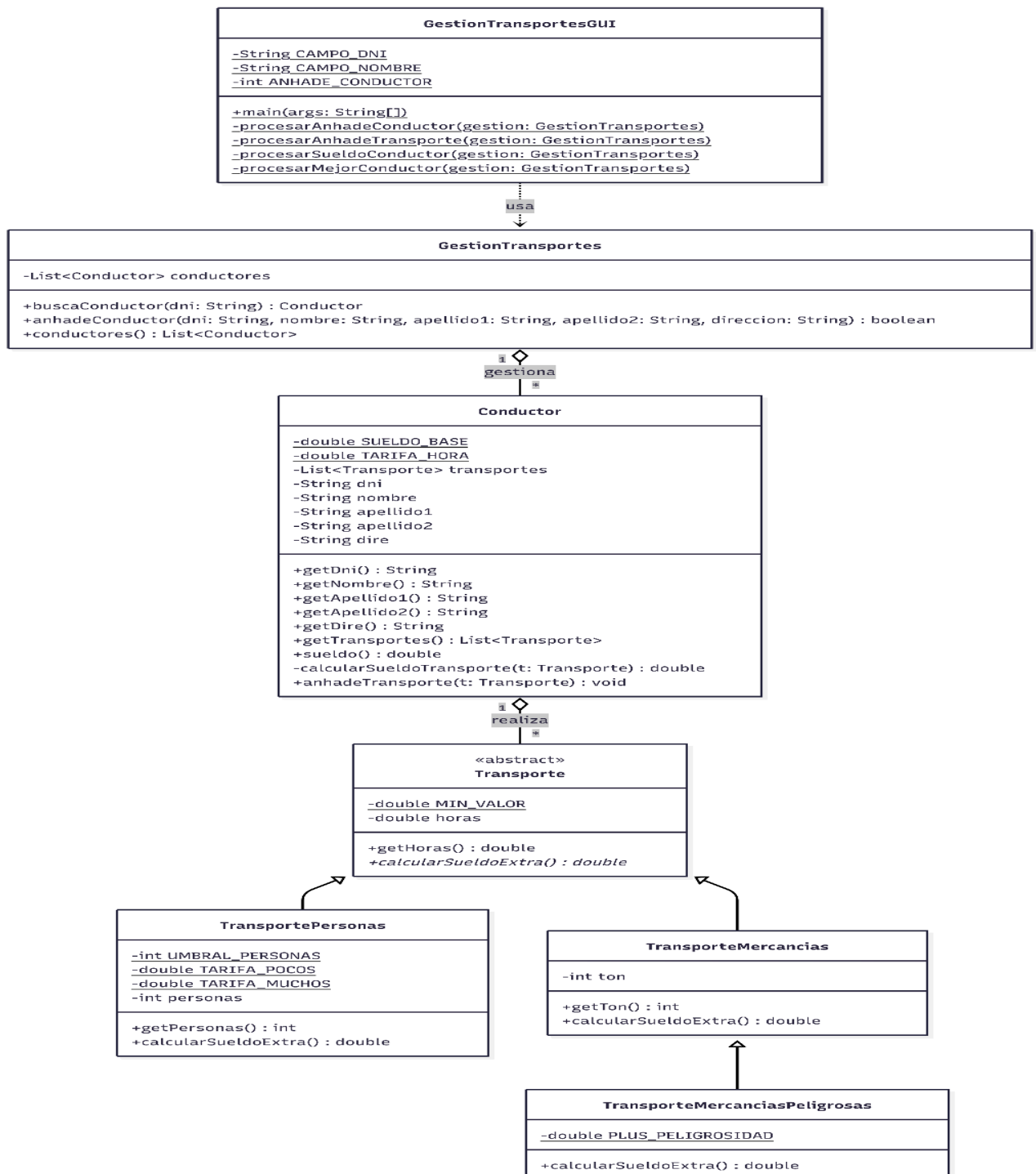
1. **Replace Type Code with Subclasses:** Se eliminó el enumerado **CategoriaTransporte** que actuaba como código de tipo. En su lugar, se convirtió **Transporte** en una clase abstracta y se crearon tres subclases concretas: **TransportePersonas**, **TransporteMercancias** y **TransporteMercanciasPeligrosas**.
2. **Replace Conditional with Polymorphism:** Consecuencia directa de la anterior. Se eliminó la sentencia **switch** que evaluaba el tipo de transporte en el método **sueldo()** de la clase **Conductor**. Ahora, el cálculo específico se cambia al método polimórfico **calcularSueldoExtra()** implementado en cada subclase de transporte.
3. **Extract Method:** \* En **GestionTransportesGUI**: Descompuse el monolítico método **main()** extrayendo la lógica de cada opción del menú a métodos privados independientes (**procesarAnhadeConductor**, **procesarAnhadeTransporte**, etc.).
  - En **Conductor**: Extraje la lógica de cálculo individual de un transporte al método privado **calcularSueldoTransporte()**.
4. **Encapsulate Collection:** En las clases **GestionTransportes** y **Conductor**, los métodos que devuelven listas de objetos (**conductores()** y **getTransportes()**) se modificaron para retornar **Collections.unmodifiableList()**.
5. **Introduce Constant:** En **GestionTransportesGUI** y **Conductor** se sustituyeron los "números mágicos" y las cadenas de texto por constantes estáticas (**static final**), como **CAMPO\_DNI**, **SUELDO\_BASE** o **TARIFA\_HORA**.
6. **Rename Method / Rename Variable:** Se actualizaron identificadores para entenderlos mejor y seguir convenciones estándar. Por ejemplo, la variable **cs** en **GestionTransportes** pasó a llamarse **conductores**, y métodos como **dni()** pasaron a **getDni()** (manteniendo los antiguos marcados como **@Deprecated** por retrocompatibilidad).

## 2. Situación Inicial vs. Situación Final del Sistema

### Situación Inicial

El sistema original presentaba un diseño anidado. La toma de decisiones estaba fuertemente centralizada: **Conductor** debía conocer todas las reglas de negocio de todos los transportes para calcular un sueldo basándose en el enumerado **CategoriaTransporte**.

### Diagrama de Clases Final:



## Situación Final

Tras la refactorización, mejoramos el código de la programación orientada a objetos. Las responsabilidades están distribuidas: cada tipo de transporte sabe cómo calcular su propio sobrecoste, y la interfaz gráfica simplemente realiza las llamadas a la lógica de negocio. El sistema ahora cumple con el principio de responsabilidad única (SRP) y está abierto a la extensión pero cerrado a la modificación (Principio Open/Closed).

## Resumen de Métricas Finales

| Clase                          | WMC | CCog | CBO | DIT | NOC |
|--------------------------------|-----|------|-----|-----|-----|
| GestionTransportesGUI          | 22  | 13   | 4   | 0   | 0   |
| GestionTransportes             | 6   | 3    | 1   | 0   | 0   |
| Conductor                      | 17  | 3    | 1   | 0   | 0   |
| Transporte (Abstracta)         | 4   | 1    | 0   | 0   | 3   |
| TransportePersonas             | 5   | 2    | 1   | 1   | 0   |
| TransporteMercancias           | 4   | 1    | 1   | 1   | 1   |
| TransporteMercanciasPeligrosas | 2   | 0    | 1   | 2   | 0   |

## 3. Análisis Razonado de las Métricas

La evolución de las métricas obtenidas justifica y refleja fielmente las mejoras de calidad arquitectónica introducidas en el código:

- **Caída de la Complejidad Cognitiva (CCog):** La mejora más notable se observa en **GestionTransportesGUI**. Al extraer los métodos, el CCog del método **main()** pasó de un crítico 24 a un simple 5. Del mismo modo, en **Conductor**, el uso de polimorfismo eliminó el

bloque condicional grande, reduciendo el esfuerzo mental (CCog) de entender el cálculo del sueldo a prácticamente 1. El código es ahora mucho más predecible y fácil de leer.

- **Aumento de la Profundidad de Herencia (DIT) y Número de Hijos (NOC):** Valores que inicialmente eran 0 han aumentado (ej. NOC de **Transporte** = 3, DIT de **TransporteMercanciasPeligrosas** = 2). En lugar de ser un síntoma negativo, en este contexto confirma el éxito de la técnica *Replace Type Code with Subclasses*. Se ha movido la "inteligencia" condicional hacia la estructura del lenguaje (jerarquía de clases). Es una herencia controlada y poco profunda que fomenta la reutilización.
- **Reducción del Acoplamiento (CBO):** La clase **Conductor** mantiene un CBO muy bajo (1), dependiendo únicamente de la clase abstracta **Transporte**. Ya no necesita conocer el enumerado ni acoplarse a todas las posibles variables y categorías del negocio. Esto aísla a **Conductor** de futuros cambios; si la empresa de transportes decide añadir un nuevo tipo (ej. "TransporteInternacional"), la clase **Conductor** permanecerá inalterada.
- **Complejidad ponderada (WMCn):** Los valores normalizados (WMC por método) han bajado en todas las clases, situándose entre 0.33 y 1.67 en la jerarquía de Transportes, y alrededor de 1.41 en **Conductor**. Esto demuestra que se ha cumplido el objetivo de tener métodos cortos, concretos y con una única responsabilidad.